■ Working with Refs & Portals

A simple, easy-to-read guide covering all key concepts

1 Managing User Input with State (Two-Way Binding)

Two-way binding means that when the user types something in an input box, it updates the state — and when the state changes, the input box also updates. Both sides stay in sync at all times.

> Think of it like a mirror: whatever you type is reflected in the state, and whatever is in the state is reflected back in the input.

- Use the **useState** hook to store the current input value.
- Bind the state value to the input's **value** prop.
- Update the state using the **onChange** event handler.
- This is called **controlled input** — React fully controls it.

> ■ *Always use two-way binding when you need to validate, transform, or react to user input in real time.*

2 Fragments

In React, every component must return **one** parent element. But sometimes you don't want to add an extra just to wrap things. That's where **Fragments** come in — they let you group elements without adding extra HTML.

- Use **<React.Fragment>** or the short form **<>...</>**.
- Fragments don't create any extra DOM nodes.
- Useful when adding extra wrapper elements would break your CSS or layout.
- You can use **key** prop only with the long form **<React.Fragment key=...>**.

> Short form: <> <h1>Hello</h1> World </>

3 Introducing Refs — Connecting & Accessing HTML Elements

A **Ref** (short for reference) lets you directly access an HTML element in the browser. Instead of going through state, you get a direct line to the actual DOM element.

- Create a ref using the **useRef()** hook.
- Attach it to an element using the **ref** prop: **ref={myRef}**.
- Access the element via **myRef.current**.
- Refs are available after the component mounts (not during the first render).

■ *useRef() returns an object like { current: null }. Once the element mounts, current points to the actual DOM node.*

4 Manipulating the DOM via Refs

Once you have a ref connected to an element, you can use it to directly read or change that element — just like you would with plain JavaScript.

- **Focus an input**: **inputRef.current.focus()**
- **Read a value**: **inputRef.current.value**
- **Scroll to an element**: **divRef.current.scrollIntoView()**
- **Play/Pause media**: **videoRef.current.play()**

Common use case: When a form loads, automatically focus the first input field using a ref — no need for state!

■ *Only manipulate the DOM via refs when React can't do it — avoid overusing refs for things state handles better.*

5 Refs vs State — What's the Difference?

Both refs and state store values — but they work very differently. Knowing when to use which one is important.

Feature	State (useState)	Ref (useRef)
Causes re-render?	Yes	No
Persists across renders?	Yes	Yes
Used for UI updates?	Yes	No
Access DOM directly?	No	Yes

■ *Simple rule: Use state when you want the UI to update. Use refs when you need to access or remember something without re-rendering.*

6 Forwarding Refs to Custom Components

By default, you can't attach a ref to your own custom components. To allow this, you need to **forward** the ref from the parent down to the actual DOM element inside.

- Wrap your component with **React.forwardRef()**.
- The component receives a second argument: **ref**.
- Pass that ref to the inner HTML element you want to expose.
- Now the parent can access the inner element through its ref.

Example: A custom <MyInput /> component can forward its ref so the parent can call .focus() on the real <input> inside.

7 useImperativeHandle — Exposing a Custom API

Instead of exposing the entire DOM element through a forwarded ref, you can choose exactly which methods or values to expose. This is done with the **useImperativeHandle** hook.

- Used together with **forwardRef**.
- You define a custom object of methods that the parent can call.
- The parent only sees what you choose to expose — not the full DOM node.
- Keeps your component's internal details private and safe.

■ *Think of it as a remote control: you give the parent only the buttons they need (play, pause) — not the whole TV's internals.*

8 Practical Example — Using Refs & State Together

A great real-world example is a **modal dialog**. You might use a **ref** to directly call the browser's built-in **showModal()** method on a <dialog> element, while using **state** to track whether it should be open or closed.

- Ref → calls **dialogRef.current.showModal()** to open the dialog natively.
- State → controls whether the component renders the dialog content.
- Both tools work together for a smooth user experience.
- This pattern avoids complex workarounds and uses browser built-ins efficiently.

Key insight: Refs handle the 'how' (directly control the element), while State handles the 'what' (what should be shown).

9 Sharing State Across Components

When two or more components need to share the same data, the best approach is to **lift the state up** to their closest common parent component.

- Move the shared state to the **parent** component.
- Pass it down to child components as **props**.
- Pass the **setter function** down if children need to update it.
- This keeps data in one place — easier to manage and debug.

Example: A shopping cart total shared between a <CartSummary /> and a <CheckoutButton /> — both receive the total from the parent.

■ *If many components need the same state, consider using React Context or a state management library like Zustand or Redux.*

10 Portals — Rendering Outside the Parent

Normally, React renders everything inside the root of your app. But sometimes — like for modals, tooltips, or notifications — you want to render content outside of that container, directly into the <body>. That's what **Portals** are for.

- Use **ReactDOM.createPortal(children, domNode)**.
- The component still behaves like normal React (events, props, context all work).
- Visually, it renders outside the parent — great for overlays and popups.
- Avoids CSS issues like **overflow: hidden** clipping your modal.

Common use: Modals, toast notifications, dropdown menus, and tooltips that need to appear on top of everything else.

■ *Even though the portal renders elsewhere in the HTML, React events still bubble up through the component tree — not the DOM tree.*